ONBOARDING.md — Введение в проект LocalScript

True Tech Hack 2026 · **Трек** 2

Этот документ поможет понять проект с самых основ — от предметной области и задачи до архитектуры и структуры кода. Читай сверху вниз. Никакого предварительного контекста не нужно.

Содержание

1. Предметная область: что такое MWS Octapi
2. Какую задачу решает LocalScript
3. Почему LLM, а не шаблоны
4. Почему локально, без облака
5. Как устроена система изнутри
6. Стек технологий: что это и зачем
7. Структура репозитория с описаниями
8. Как запустить (кратко)
9. Среда Lua MWS: что нельзя и что можно
10. Известные особенности и подводные камни
11. Полезные ссылки

1. Предметная область: что такое MWS Octapi

MWS Octapi — корпоративная платформа автоматизации бизнес-процессов. Бизнес-аналитики создают в ней «сценарии» (workflows): последовательности шагов, ветки условий, циклы. Каждый шаг может выполнять небольшой Lua-скрипт, который что-то делает с данными рабочего процесса.

Данные рабочего процесса доступны через специальный объект `wf`:

```
wf.vars.orders           -- массив заказов текущего процесса
wf.vars.userName         -- строковое поле
wf.initVariables.limit   -- параметр инициализации
```

Это не стандартный Lua — это закрытая изолированная среда.
Нельзя использовать `require`, `io`, `os` и любые внешние библиотеки.
Массивы создаются только через `_utils.array.new()`.
Каждый скрипт должен заканчиваться оператором `return`.

Зачем вообще Lua? MWS Octapi использует Lua как встраиваемый скриптовый язык для пользовательской логики — он лёгкий, быстрый и легко изолируется.

2. Какую задачу решает LocalScript

Бизнес-аналитики не умеют программировать. Им нужно описать задачу словами, а получить готовый Lua-код, который можно скопировать и вставить в шаг платформы.

LocalScript — HTTP API. Принимает текстовый запрос, возвращает Lua-код:

```
Запрос:
  POST /generate
  {"prompt": "отфильтровать заказы со статусом выполнен"}

Ответ:
  {"code": "local result = _utils.array.new()\nfor _, o in ipairs(wf.vars.orders) do
```

Дополнительно поддерживается поле `context` — существующий Lua-код, который нужно доработать:

```
{
  "prompt": "добавь проверку: если массив пустой — вернуть nil",
  "context": "local result = _utils.array.new()\nfor _, o in ipairs(wf.vars.orders)
}
```

3. Почему LLM, а не шаблоны

Разные аналитики описывают одно и то же по-разному:

Что написал аналитик	Что он хочет
«оставить только выполненные заказы»	filter(orders, status == done)
«убрать из списка незавершённые заявки»	filter(orders, status == done)
«выбрать orders где status = готово»	filter(orders, status == done)
«отфильтруй завершённые»	filter(orders, status == done)

Шаблонный движок ищет точное совпадение строк — он не найдёт ни одного из этих вариантов.

LLM (Large Language Model) понимает **смысл** запроса, а не его буквальную форму. Это ключевое свойство для данной задачи.

Помимо этого, скрытая тестовая выборка хакатона включает задачи на **доработку существующего кода**. Шаблонный движок не может изменить имеющийся код в ответ на текстовую инструкцию. LLM — может, через поле `context`.

4. Почему локально, без облака

Требования трека явно запрещают любые внешние API-вызовы во время работы системы.
Система должна работать внутри защищённого контура без выхода в интернет.

Мы используем:

- Ollama — инструмент для запуска open-source LLM на собственном железе.
  Работает как HTTP-сервер (порт 11434). Загружает веса модели в GPU VRAM и отвечает на запросы.
- Qwen2.5-Coder:7b-instruct-q4_K_M —
  open-source LLM от Alibaba, специализированная на генерации кода, поддерживает 29+ языков включая русский, помещается в 8 ГБ VRAM в формате Q4-квантизации (~4.7 ГБ).

5. Как устроена система изнутри

```
Ты / платформа MWS Octapi
        |
        |  POST http://localhost:8080/generate
        |  {"prompt": "...", "context": "..."}
        ▼
FastAPI (uvicorn, порт 8080)       ← контейнер api: Python-код
        |
        |  1. PromptBuilder собирает промпт
        |     system_prompt.txt + 20 few-shot примеров + context (если есть) + prompt
        |
        |  2. OllamaClient отправляет промпт
        |     httpx POST → ollama:11434/api/generate
        ▼
Ollama (порт 11434)                ← контейнер ollama: LLM на GPU
        |  Модель думает, генерирует <thinking> + <code>
        ▼
FastAPI снова:
        |  3. CoT-парсер: извлечь <code>...</code>
        |  4. LuaValidator: luac -o /dev/null - (проверка синтаксиса)
        |  5. Если ошибка — добавить её в промпт и повторить (макс. 3 раза)
        ▼
{"code": "return wf.vars..."}      ← чистый Lua без markdown-обёрток
```

Что такое Chain-of-Thought (CoT)?

Это техника промптинга: модели предлагают сначала написать рассуждение,
а только потом — код. Системный промпт явно инструктирует модель:

```
<thinking>
1. Перевести задачу на английский
2. Пошагово спланировать Lua-логику
</thinking>
<code>
```

```
[только чистый Lua]
</code>
```

Токены рассуждения, записанные перед кодом, служат моделе контекстом для собственной генерации — она буквально «думает вслух». API возвращает только `<code>`, рассуждение остаётся внутри.

Что такое few-shot prompting?

В системный промпт включены 20 готовых примеров пар «задача → Lua-код» для задач платформы MWS. Модель видит эти примеры и понимает ожидаемый стиль и формат ответа ещё до того, как получает реальный запрос пользователя.

Файл `prompts/system_prompt.txt` — **главный рычаг для улучшения точности системы**.

6. Стек технологий: что это и зачем

Технология	Роль в проекте	Ссылка
Ollama	Запуск LLM локально, HTTP-сервер на порту 11434	https://ollama.com/docs
Qwen2.5-Coder 7B	Сама языковая модель, генерирует код	https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct
FastAPI	Python HTTP-фреймворк, реализует эндпоинты API	https://fastapi.tiangolo.com/ru/
uvicorn	ASGI-сервер, запускает FastAPI	https://www.uvicorn.org
httpx	Python HTTP-клиент для запросов к Ollama	https://www.python-httpx.org
Pydantic v2	Валидация и сериализация запросов/ответов	https://docs.pydantic.dev/latest/
Docker Compose	Запускает api + ollama вместе одной командой	https://docs.docker.com/compose/
lua5.4 / luac	Синтаксическая проверка сгенерированного Lua	https://www.lua.org/manual/5.4/

7. Структура репозитория с описаниями

```
task-repo/
│
├── README.md
│   Техническая документация. Архитектура C1/C2/C3, запуск,
│   переменные среды, API-справка, известные ограничения.
│
├── ONBOARDING.md              ← этот файл
│   Введение с нуля. Объясняет зачем, что и как — от основ до архитектуры.
│
```

```
├── Dockerfile
│   Образ для контейнера api.
│   python:3.11-slim + lua5.4 (для luac) + Python-зависимости.
│
├── docker-compose.yml
│   Запускает два контейнера: api (порт 8080) и ollama (порт 11434).
│   Между ними — внутренняя Docker-сеть. Модель хранится в volume ollama-data.
│
├── requirements.txt
│   Python-зависимости: fastapi, uvicorn[standard], pydantic, httpx.
│
├── api/
│   ├── __init__.py           — пустой файл, делает директорию пакетом Python
│   ├── main.py               — FastAPI app: /generate, /health, lifespan warm-up
│   ├── models.py             — Pydantic: GenerateRequest (prompt + context), Gener
│   ├── agent.py              — AgentPipeline: оркестратор всего процесса генерации
│   │                           OllamaClient, CoT-парсер, retry loop
│   ├── validator.py          — LuaValidator: вызывает luac, возвращает (ok, stderr)
│   └── prompt_builder.py      — сборка финального промпта из частей
│
├── prompts/
│   └── system_prompt.txt
│       САМЫЙ ВАЖНЫЙ ФАЙЛ.
│       CoT-шаблон + 20 few-shot примеров для задач MWS Octapi.
│       Изменение этого файла — основной способ улучшить точность системы.
│
├── docs/
│   ├── api_contract_description.md
│   │   Подробное описание HTTP API: все поля запросов и ответов,
│   │   примеры для curl/Python/PowerShell, кодировочная проблема PS 5.1.
│   │
│   ├── DEBUGGING.md
│   │   Все известные баги, с которыми мы столкнулись, и их решения.
│   │   Рабочие процессы для отладки промптов и поведения модели.
│   │
│   └── architecture/
│       Диаграммы C4: по одному файлу на каждый уровень.
│       c1_context.md, c2_containers.md, c3_components.md
│
└── tests/
    ├── test_base_cases_data.py
    │   Единый словарь тест-кейсов (источник правды).
    │   Для каждого случая: эталонный Lua-код + список перефразировок.
    │   Именно отсюда тест-раннер берёт «правильный ответ».
    │
    ├── test_base_cases.py
    │   Тест-раннер на базе unittest.
    │   Запускает API, отправляет все перефразировки, сравнивает с эталоном,
    │   показывает прогресс в реальном времени, сохраняет отчёт.
    │
    └── reports/
        Автоматически сохранённые отчёты после каждого запуска тестов.
        Форматы: .md (человекочитаемый) и .json (машиночитаемый).
        Папка добавлена в .gitignore.
```

8. Как запустить (кратко)

Через Docker (рекомендуется)

```
# 1. Скачать модель (один раз):
docker-compose run --rm ollama ollama pull qwen2.5-coder:7b-instruct-q4_K_M

# 2. Запустить всё:
docker-compose up --build

# 3. Проверить:
curl -X POST http://localhost:8080/generate \
     -H "Content-Type: application/json" \
     -d '{"prompt":"получить последний email из списка"}'
```

На Windows без Docker (для разработки)

Подробная инструкция с описанием всех нюансов — в `README.md` (раздел 8) и в `docs/DEBUGGING.md`.

9. Среда Lua MWS: что нельзя и что можно

✅ Можно

```
-- Обращаться к переменным через wf.vars и wf.initVariables
local orders = wf.vars.orders
local limit  = wf.initVariables.maxItems

-- Создавать массивы через _utils.array.new()
local result = _utils.array.new()

-- Использовать стандартные Lua-функции: string, math, table, ipairs, pairs
for _, item in ipairs(orders) do
  table.insert(result, item)
end

-- Заканчивать скрипт оператором return
return result
```

❌ Нельзя

```
require("something")   -- запрещено
io.read()             -- запрещено
os.execute("...")      -- запрещено
```

10. Известные особенности и подводные камни

PowerShell 5.1 и кириллица

PowerShell 5.1 (встроенный в Windows 10/11) кодирует тело HTTP-запроса в ASCII. Кириллические символы превращаются в `????????`, и модель «не понимает русский».

Это не баг модели — Qwen2.5-Coder поддерживает русский язык.
Это баг среды отладки. Решение: передавать тело как UTF-8 байты или использовать PS 7+.

Подробно — в `docs/DEBUGGING.md`.

Системный прокси (Clash Verge, Proxifier)

При включённом системном прокси Python направляет все запросы через него, включая запросы к `localhost:11434`. Прокси возвращает `502 Bad Gateway`.

Решение: `$env:NO_PROXY = "localhost,127.0.0.1,::1"` перед запуском uvicorn.

KEEP_ALIVE и выгрузка модели из VRAM

По умолчанию Ollama выгружает модель из VRAM через 5 минут простоя.
При локальной разработке добавьте `$env:OLLAMA_KEEP_ALIVE = "-1"` перед `ollama serve`.

DRY_RUN на Windows

`luac` не входит в стандартную поставку Windows.
При `DRY_RUN=true` валидация синтаксиса пропускается.
В Docker-контейнере `lua5.4` установлен, и `DRY_RUN=false` работает корректно.

11. Полезные ссылки

Тема	Ссылка
Ollama — что это и как работает	https://ollama.com/docs
Qwen2.5-Coder — модель	https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct
FastAPI	https://fastapi.tiangolo.com/ru/
Few-shot prompting	https://www.promptingguide.ai/ru/techniques/fewshot
Chain-of-Thought prompting	https://www.promptingguide.ai/ru/techniques/cot
Lua 5.4 — справочник	https://www.lua.org/manual/5.4/
Docker Compose	https://docs.docker.com/compose/
httpx (Python HTTP клиент)	https://www.python-httpx.org/
Pydantic v2	https://docs.pydantic.dev/latest/